

Towards Machine Learning for Placement and Routing in Chip Design: a Methodological Overview

Junchi Yan^{1*}, Xianglong Lyu¹, Ruoyu Cheng¹, Yibo Lin^{2*}

¹Department of CSE, and MoE Key Lab of Artificial Intelligence, Shanghai Jiao Tong University

²Department of EECS, Peking University

{yanjunchi,kyle_lyu,roy_account}@sjtu.edu.cn yibolin@pku.edu.cn

Abstract

Placement and routing are two indispensable and challenging (NP-hard) tasks in modern chip design flows. Compared with traditional solvers using heuristics or expert-well-designed algorithms, machine learning has shown promising prospects by its data-driven nature, which can be of less reliance on knowledge and priors, and potentially more scalable by its advanced computational paradigms (e.g. deep networks with GPU acceleration). This survey starts with the introduction of basics of placement and routing, with a brief description on classic learning-free solvers. Then we present detailed review on recent advance in machine learning for placement and routing. Finally we discuss the challenges and opportunities for future research.

1 Introduction

The scale of integrated circuits (ICs) has increased dramatically, posing a challenge to the scalability of existing Electronic Design Automation (EDA) techniques and technologies. The increasing circuit density incurs additional issues for very large scale integration (VLSI) placers and routers as the feature size of modern VLSI design continues to drop and on-chip connectivity gets increasingly sophisticated. Due to increased on-chip connectivity, concentrated needs, and restricted resources, modern designs are prone to congestion issues and wire-length minimization, which has become a critical task at every stage of the design process. Accordingly, placement and routing – the elements of the design cycle physically arranges the locations and the courses of nets – becomes more crucial in modern VLSI.

Placement and routing are two of the most critical but time-consuming steps of the chip design process. Placement assigns various logic components like logic gates and functional blocks into discrete sites in the physical layout of a chip, and routing finishes the physical wiring of interconnections. The goal of placement and routing is to optimize power, performance, and area (PPA) metrics within constraints, e.g., placement density and routing congestion.

*Correspondence author.

Placement can be regarded as a much more complicated variation of 2D bin packing problem with geometrical constraints. The latter is known to be NP-hard [Hartmanis, 1982]. The objective of placement is correlated to both the logical interconnection of the circuit design and the geometrical locations of logical components. As the quality of a placement solution cannot be accurately evaluated until routing, resulting in a long feedback loop in the design flow, modern placement needs to reduce routing congestion and improve routability in early stage. Routing has been proven to be an NP-hard problem [Hartmanis, 1982], even in the simplest case with only a few two-pin networks. Given the placement solution, it connects the pins of millions of networks with physical wires subjecting to the limited routing resources and complicated geometrical design rules. Routing is tightly coupled with placement and an excellent placement solution leads to better chip area utilization, timing performance, and routability.

There are also surveys on routing and placement. [Huang *et al.*, 2021] summaries a comprehensive review of existing ML studies for EDA field, most of which belongs to the four categories: decision making in traditional methods, performance prediction, black-box optimization, and automated design, ordered by increasing degree of automation. Although it covers various stages in the EDA flow, it provides shallow analysis for specific tasks, placement and routing, compared to ours. [Hamolia and Melnyk, 2021] introduces the ISPD 2015 dataset for comparison between classic methods and ML algorithms. [Rapp *et al.*, 2021] categorizes how ML may be used and is used for design-time/run-time optimization and exploration strategies of ICs, along with trends in the employed ML algorithms. [Markov *et al.*, 2015] reviews the history of placement research and the progress leading up to the state of the art. However, they only consider classic placer in the past 50 years, which is complementary to our survey which focuses on learning-based placer in recent years.

2 Problem Background and Classic Solvers

This section starts with the problem formulation and mainstream solvers which are traditionally learning-free. We then introduce recent applicable learning techniques.

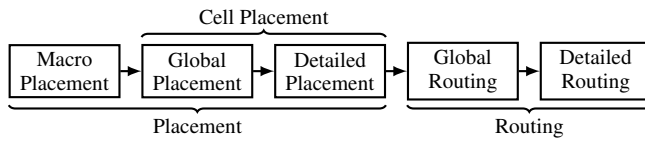


Figure 1: Example of a simplified placement and routing flow.

2.1 Placement Problem

Placement can be performed at different levels. In general, the global placement involves macro placement and standard cell placement, as shown in Figure 1. The detailed placement includes legalization, wirelength and routability refinement.

Global Placement. It is one of the most crucial but time-consuming steps in the chip design process, which can be cast as a constrained optimization problem. It assigns exact locations for various components of a netlist including macros and standard cells within the chip layout. Standard cells are basic logic cells, i.e., logic gates, and macros are pre-designed IP blocks, e.g., SRAMs. A good placement leads to better chip area utilization, timing performance, and routability, while inferior placement assignment will affect the chip's performance and even make it nonmanufacturable.

Formally, the input of global placement is a netlist that can be represented by hypergraph $H = (V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ denotes set of nodes (cells), and $E = \{e_1, e_2, \dots, e_m\}$ denotes set of hyperedges (nets). Hyperedge $e_i \in E$ is a subset of nodes. We seek to determine locations of macros and standard cells (x_i, y_i) which are further combined into two vectors $\vec{x} = (x_1, \dots, x_n)$ and $\vec{y} = (y_1, \dots, y_n)$. Wirelength minimization is one of the main objectives for placement. $HPWL_N(\vec{x}, \vec{y})$ is usually adopted to approximate wirelength during placement as:

$$HPWL = \sum_{e_i \in E} \left(\max_{v_j \in e_i} x_j - \min_{v_j \in e_i} x_j + \max_{v_j \in e_i} y_j - \min_{v_j \in e_i} y_j \right)$$

Detailed Placement. Solution from global placement is often illegal: cells may overlap or occupy illegal sites, e.g. between placement rows. This is because global placers are unaware of these constraints. Detailed placement performs legalization and quality refinement. The legalization removes overlaps between cells and snaps them to the sites of rows with minimum adverse impact on placement quality. The refinement then takes a legal placement and further improves objectives wirelength and routability by locally moving cells.

2.2 Routing Problem

Routing often involves two cases: global and detailed routing.

Global Routing connects wires with metal resources on a grid graph $G(V, E)$ representing the physical layout of a chip. Essentially, the physical layout is divided into rectangular areas with each area corresponding to a global routing cell (G-Cell), denoting each vertex $v_i \in V$. Each edge $e_{ij} \in E$ represents the joint boundary between abutting G-Cells v_i and v_j . The capacity for an grid edge e , c_e , is defined as the maximum number of wires that can cross the grid edge. The usage u_e is defined as the actual number of wires crossing the grid edge. The overflow o_e is defined as $\max(0, u_e - c_e)$. When

multiple metal layers are adopted for routing, a vertical dimension is introduced to the grid graph, where abutting metal layers are connected through vias in the 3D grid graph.

Detailed Routing works on a fine-grained 3D grid graph compared with global routing, considering complicated design rules. Each grid edge provides unit routing resource such that a detailed routing path on the grid does not violate the design rules like minimum metal width and spacing. Detailed routing also needs to consider preferred routing directions, where abutting routing layers prefer perpendicularly routing directions. Consecutive grid edges following the preferred direction (e.g., in X or Y direction) of their corresponding layer are described as a routing track. Typical detailed routing algorithms take global routing segments as guidance. Each global routing segment provides rough hints for the topologies of nets, and detailed routing needs to finalize the actual routing paths on the fine-grained grid graph. Nets unable to be routed without conflicts on the grid graph can cause design rule violations, which is a widely-used metric to describe the quality of detailed routing solutions or how congested a design is.

2.3 Classic Placement Solvers

The history of VLSI placement can trace back to the 1960s [Breuer, 1977; Fiduccia and Mattheyses, 1982], when partition-based methods adopted the idea of divide-and-conquer: netlist and chip layout are partitioned recursively until the sublist can be solved by optimal solvers. This hierarchical structure makes them fast to execute and natural to extend for larger netlists at the cost of solution quality, since each sub-problem is solved independently. Multi-level partitioning methods based on Fiduccia–Mattheyses heuristics [Agnihotri *et al.*, 2005; Caldwell *et al.*, 2000] were developed afterwards. Analytical placers appeared in the early 1980s, but were eclipsed by annealing methods [Kirkpatrick *et al.*, 1983] inspired from annealing in metallurgy that involves heating and controlled cooling for optimal crystalline surfaces. In practice, simulated annealing (SA) optimizes a given placement solution by random perturbations with actions such as shifting, swapping and rotation of macros [Ho *et al.*, 2004; Shunmugathammal *et al.*, 2020]. Although SA is flexible and able to find the global optimum, it is time-consuming and hard to deal with the ever-increasing scale of circuit. Later, analytical techniques have matured including force-directed methods [Spindler *et al.*, 2008] and non-linear optimizers [Chen *et al.*, 2008b; Kahng and Wang, 2005]. In comparison, the quadratic methods are efficient but show relatively worse performance, while non-linear optimization approximates the cost functions more smoothly with the cost of higher complexity. Recently, however, modern analytical placers e.g. ePlace [Lu *et al.*, 2015] and RePlace [Cheng *et al.*, 2018a] introduce electrostatics-based global-smooth density cost function and Nesterov's method nonlinear optimizer that achieve superior performance on public benchmarks. They formulate each node of the netlist as positively charged particle. Nodes are adjusted by their repulsive force and the density function corresponds to system's potential energy. These analytical methods update positions of cells in gradient based optimization scheme and gen-

erally can handle millions of standard cells by parallelization on multi-threaded CPUs using partitioning to reduce the run time.

With increasing design complexity, optimizing traditional placement metrics (HPWL) alone is insufficient in practice [Alpert *et al.*, 2010]. Therefore, routability-driven placers are proposed to emphasize on routing failures. SimPLR [Kim *et al.*, 2011] develops lookahead routing to temporarily bloat cells, modulate target aspect ratio of cell placement areas, and modify anchor positions during quadratic placement. NTUPlace4 [Hsu *et al.*, 2011] adopts probabilistic congestion estimation when modeling pin density. There are also other optimization directions such as timing-driven placement for optimizing circuit delay [Riess and Ettelt, 1995; Luo *et al.*, 2006] and IC power optimization for circuit power [Cheon *et al.*, 2005; Lee and Markov, 2012]. Another trend is the increasingly extensive use of intellectual property (IP) modules and pre-designed macro blocks. As a result, mixed-size placement tools are becoming indispensable for physical design. [Taghavi *et al.*, 2005] proposes a hierarchical method to place large scale mixed size designs that may contain thousand of macro blocks and millions of standard cells, based on min-cut partitioning and simulated annealing which are both aware of large macro cells. [Chen *et al.*, 2008a] designs a novel constraint graph-based macro placement algorithm that removes macro overlaps and optimizes macro positions and orientations effectively and efficiently for modern mixed-size circuit designs.

2.4 Classic Routing Solvers

Global Routing Solvers

The global routing approaches can be divided into two types: concurrent and sequential. The sequential approach has been proved to be very effective in practice and considerably faster than the concurrent approach, but it highly relies on the ordering of the nets and thus, being prone to a sub-optimal solution.

Concurrent approaches attempt to handle numerous nets simultaneously but are typically too expensive to be applied on today's large designs, which may contain up to a million nets. BoxRouter [Cho and Pan, 2007] implements progressive integer linear programming (ILP) and adaptive maze routing to effectively diffuse the congestion. However, its progressive ILP routing formulation only covers L-shape patterns, and it fails in difficult scenarios when most nets must be detoured in complex patterns. On top of it, BoxRouter 2.0 [Cho and Pan, 2007] further provides more powerful and systematic way of eliminating congestion and assigning layers to wires. GRIP [Wu *et al.*, 2009; Wu *et al.*, 2010] is based on a partitioning strategy in a full 3D manner and obtains the best wirelength among the open literature, but when compared to other recent global routers, GRIP requires prohibitive overall runtime.

Sequential approaches often use net decomposition [Chu and Wong, 2007], maze routing [Lee, 1961], pattern routing [Kastner *et al.*, 2002], or negotiation-based rip-up and rerouting (NRR), and only one net is routed at a time. Archer [Ozdal and Wong, 2009] explores the congestion histories, and adopts a Lagrangian relaxation-

based bounded-length min-cost topology improvement algorithm that enables Steiner trees to change dynamically for congestion optimization. NTUgr [Chen *et al.*, 2009] replaces iterative NRR by enhanced iterative forbidden-region rip-up/rerouting (IFR). FastRoute [Pan *et al.*, 2012], on the other hand, integrates several novel techniques: fast congestion-driven via-aware Steiner tree construction, 3-bend routing, virtual capacity adjustment, multisource multi-sink maze routing, and spiral layer assignment. NTHU-Route 2.0 [Chang *et al.*, 2010] improves NTHU-Route [Gao *et al.*, 2008], an earlier version by a new history-based cost function and new ordering methods for congested region identification and rip-up and reroute. NCTU-GR 2.0 [Liu *et al.*, 2013] outperforms the foregoing global routers by applying two bounded-length maze routing (BLMR) algorithms (i.e. optimal-BLMR and heuristic-BLMR), a rectilinear Steiner minimum tree aware routing scheme, a collision-aware rip-up and rerouting scheme and a 3-D wire length optimization technique.

Detailed Routing Solvers

Since the 1970s, detailed routing has been extensively researched (e.g. [Yoshimura and Kuh, 1982]), and rip-up and reroute, such as the one in Mighty [Shin and Sangiovanni-Vincentelli, 1987] has been the most common technique for detailed routing. Nevertheless, when dealing with congested designs, such a sequential net-by-net method is unproductive and frequently results in unneeded detours. DUNE [Cong *et al.*, 2001] and MR [Chang and Lin, 2004] develop multilayer techniques to handle full-chip gridless routing, in which the routing passes through a coarsening and uncoarsening phase. These multilevel routers, however, continue to use the sequential rip-up and reroute technique. Several attempts have been made to evaluate nets more concurrently during detailed routing. Based on Boolean satisfiability, [Nam *et al.*, 2002] suggests a thorough FPGA router which delivers exceedingly long runtime despite good solution quality. Track assignment is introduced in [Batterywala *et al.*, 2002] as a step between global and detailed routing. Segments taken from the global routing solution are assigned to routing tracks in track assignment. [Ozdal, 2009] introduces an ingenious method for doing escape routing for dense pin clusters, which is a major bottleneck in detailed routing. However, the technique is not recommended for solving detailed routing on a whole-chip scale. Routing frameworks based on rules are well-suited to traditional design flow and have been in use for decades. To execute correct pin access, [Nieberg, 2011] computes various pin access path candidates and selected the shortest paths from pins to grid points that did not violate any design criteria. RegularRoute [Zhang and Chu, 2011] frames the global segment assignment problem inside each group of routing tracks as a maximum weighted independent set problem, then used regular routing patterns in a bottom-up layer-by-layer framework. Under self-aligned double patterning limitations, [Xu *et al.*, 2016] suggests a pin access-driven rip-up and reroute scheme. For mixed-cell-height circuits, [Li *et al.*, 2018] introduces a pin access-aware legalizing technique. Due to the spirited

ISPD'18 and ISPD'19 routing contest [Mantik *et al.*, 2018; Liu *et al.*, 2019], some new work has been completed. [Kahng *et al.*, 2018] divides each layer into parallel panels and expressed the routing problem as an integer linear program on each panel. [Sun *et al.*, 2018] modifies the notion of hit points and employs through violations to assign tracks. Dr. CU [Chen *et al.*, 2019] proposes an algorithm for finding the best path while keeping the minimum-area requirement in mind. The subsequent Dr. CU 2.0 [Li *et al.*, 2019] handles hard-to-access pins and new design rules including length-dependent parallel run length spacing, end-of-line spacing with parallel edges, and corner-to-corner spacing.

3 Machine Learning for Placement

3.1 Traditional Placers Enhancement

Most traditional placers mentioned above perform heavy numerical computation for large-scale optimization problem on the CPUs, which lacks exploration of GPU's opportunity. DREAMPlace [Lin *et al.*, 2020] is inspired by the idea that the analytical placement problem is analogous to training a neural network. They both involve optimizing parameters and minimizing a cost function. Based on the state-of-the-art analytical placement algorithm RePlace, DREAMPlace implements hand-optimized key operators by deep learning toolkit PyTorch and achieves over $30\times$ speedup against CPU-based tools. PL-GNN [Lu *et al.*, 2021] presents a graph learning-based framework that provides placement guidance for commercial placers by generating cell clusters based on logical affinity information and attributes of design instances. PADE [Ward *et al.*, 2012] improves data path logic through automatic data path extraction and evaluation, in which the placement of data path logic is conducted separately from random logic. [Agnesina *et al.*, 2020] proposes a deep reinforcement learning (RL) framework to optimize the placement parameters of commercial EDA tool. An agent learns to tune parameters autonomously, trained solely by RL from self-search. Handcrafted features along with graph embeddings generated using unsupervised Graph Neural Networks are adopted for generalization to unseen netlists.

3.2 Placement Decision Making

Learning-based methods for placement decision especially RL have been proposed to obtain the generalization ability. Existing RL applications have demonstrated the effectiveness on macro placement, where there are typically fewer than 1000 macros to place. Google [Mirhoseini *et al.*, 2021] proposes an end-to-end learning method for macro placement that models chip placement as a sequential decision making problem. In each step, the RL agent places one macro and target metrics are used as reward until the last action. GNN is adopted in the value network to encode the netlist information and deconvolution layers in the policy network output the mask of current macro position. DeepPlace [Cheng and Yan, 2021] first proposes a joint learning technique for the placement of macros and standard cells by the integration of reinforcement learning with a gradient-based classical cell placer ([Lin *et al.*, 2020]). To further bridge the

placement with the subsequent routing task, they also develop a joint learning approach DeepPR via RL to fulfill both placement and routing for macros. Table 2 compares learning-based placers in terms of placement target and learning protocol.

3.3 Prediction Model Embedded in Placement

ML also assists placers to optimize complicated objectives like routability by embedding prediction models, as it is difficult to foresee routing congestion accurately during placement. [Huang *et al.*, 2019] proposes the first routability driven macro placement with deep learning. A CNN-based routability prediction model is proposed and embedded into a macro placer such that a good macro placement with minimized design rule check (DRC) violations can be derived through SA optimization process. [Chan *et al.*, 2017] presents a learning based algorithm to predict DRC violations in detailed routing and automatically improve the routability of these designs. [Liu *et al.*, 2021b] predicts congestion hotspots and then incorporates this prediction model into a placement engine, showing how an ML-based routing congestion estimator can be embedded into the global placement stage.

3.4 Challenges and Limitations for Placement

The major challenges for ML applications in placement lie in two folds: long feedback loop and high requirement of scalability. Placement objectives like routability cannot be evaluated until routing finished; hence, it may take hours to obtain the feedback in the optimization loop, which is unaffordable to make thousands of queries. Modern placers need to handle tens of thousands of macros and millions of standard cells within several hours. Such requirement of scalability is still beyond the capability of existing ML approaches.

4 Machine Learning for Routing

4.1 Learning-aided Routability Prediction

In the placement step, the essential requirements of routing design guidelines must be considered. However, it is difficult to precisely and quickly estimate routing information during the placement step, and researchers have lately used machine learning to overcome this problem. Table 1 summarizes the recent efforts on routability prediction, which can be categorized into congestion count prediction and congestion location prediction at different design stages. Congestion count denotes congestion related metrics such as total congestion and number of design rule violations, while congestion locations require detailed locations of congestion or design rule violations, usually represented as a 2D map.

Congestion count. As mentioned above, it denotes the overall amount of routing congestion, useful to evaluate how good a placement solution is. Efficient prediction of congestion count can reduce the turn-around time in the design flow by avoiding the time-consuming routing stage. [Qi *et al.*, 2014; Zhou *et al.*, 2019] capture multiple factors in global routing and enable prediction of detailed routing congestion using multivariate adaptive regression splines (MARS). [Tabrizi *et al.*, 2018; Maarouf *et al.*, 2018] attempt

Task type	Feature from	Label at	Publication	Backbone models	Benchmark
Congestion Count (Scalar or Vector as Labels)	Global Routing	Detailed Routing	[Qi <i>et al.</i> , 2014]	MARS	ASIC
			[Zhou <i>et al.</i> , 2019]	MARS	
	Cell Placement	Global Routing	[Tabrizi <i>et al.</i> , 2018]	MLP	ASIC
			[Maarouf <i>et al.</i> , 2018]	LR, RF, MLP	FPGA
	Cell Placement	Detailed Routing	[Zhou <i>et al.</i> , 2015]	MARS	ASIC
			[Chan <i>et al.</i> , 2016]	MARS, SVM	
Congestion Locations (2D Map as Labels)	Macro Placement	Global Routing	[Cheng <i>et al.</i> , 2018b]	LR, RF, Boosting, MLP	ASIC
	Cell Placement	Global Routing	RouteNet [Xie <i>et al.</i> , 2018]	FCN	ASIC
			PROS [Chen <i>et al.</i> , 2020]	FCN	ASIC
			[Pui <i>et al.</i> , 2017]	LR, SVM	FPGA
			[Yu and Zhang, 2019]	Conditional GAN	
			[Alawieh <i>et al.</i> , 2020]	Conditional GAN	
			DLRoute [Al-Hyari <i>et al.</i> , 2021]	CNN	
	Cell Placement	Detailed Routing	J-Net [Liang <i>et al.</i> , 2020]	U-Net	ASIC

Table 1: Summary of recent publications on learning-aided routability prediction in terms of congestion count and locations.

to predict global routing congestion at placement stage with linear regression (LR), random forest (RF), and MLP models for datasets from ASIC and FPGA. Other studies like [Zhou *et al.*, 2015; Chan *et al.*, 2016] aim at predicting the congestion count at detailed routing given cell placement, and [Cheng *et al.*, 2018b] tries to predict the congestion count at global routing given only macro placement.

Congestion location. Its accurate prediction is necessary to effectively guide the placement and routing optimization, as it can help reserve enough space for congested regions. RouteNet [Xie *et al.*, 2018] is the first attempt to utilize CNN to forecast the locations of design rule checking (DRC) hotspots given cell placement and global routing information. A customized fully convolutional network (FCN) is constructed taking features like rectangle uniform wire density (RUDY), as a pre-routing congestion estimator, and global routing congestion map. Predicting global routing congestion locations at placement is helpful to guide routability optimization in early stages. Thus, PROS [Chen *et al.*, 2020], [Pui *et al.*, 2017], [Yu and Zhang, 2019], [Alawieh *et al.*, 2020], and DLRoute [Al-Hyari *et al.*, 2021] attempt to learn the correlation between congestion locations at global routing and cell placement. As both the features and labels can be represented as image-like tensors, many studies transform the problem into image generation tasks and leverage FCN and conditional GAN to build the correlation. [Liang *et al.*, 2020] moves one step further to directly predict congestion locations at detailed routing (i.e., locations of design rule violations) from cell placement. A customized CNN architecture, J-Net (an extension of U-Net architecture), is proposed as opposed to a plug-in use of machine learning modules. This work converts the density of pins and macros in placement results into images and optimizes an encoder-decoder model using a pixel-wise loss function. The network outputs a heat map, showing where detailed routing congestion might occur.

Among learning-aided prediction tasks, the more stages to skip in Fig. 1, the more difficult the tasks are, as we need to build models correlating with more stages. Thus, the accuracy requirement varies from task to task. Predicting congestion locations at detailed routing at early stages like cell placement or even macro placement can expedite the design.

Publication	Target for placement		Learning protocol	
	Macro	Standard cell	Module	Reward using
[Cheng and Yan, 2021]	RL	DNN backprop	CNN+GNN	cell placement
[Mirhoseini <i>et al.</i> , 2021]	RL	NA	GNN	macro placement
[Vashisht <i>et al.</i> , 2020]	RL	NA	MLP	macro placement
[He <i>et al.</i> , 2020]	RL	NA	MLP	macro placement
[Lin <i>et al.</i> , 2020]	heuristic	DNN backprop	NA	NA

Table 2: Comparison of learning-based placers. RL is the most popular paradigm for macro placement, while learning for standard cell placement still relies on traditional placers for further enhancement. Note that [Cheng and Yan, 2021] is a joint learning approach for solving placement of macro and standard cells, whereby the two kinds of components are sequentially arranged by reinforcement learning and neural network formed gradient optimization respectively and reward is based on the full placement result. [Lin *et al.*, 2020] is not a RL method, thus there is no reward function.

4.2 Deep Neural Networks for Routing

Rather than the methods mentioned above predicting the congestion information of placement, some works employ DNNs to directly handle routing problems, with a little or even without the assistance of traditional routing techniques.

[Jain and Okabe, 2017] presents a fully convolutional neural network learning to route a circuit layout with appropriate choices of metal tracks and wire class combinations. Encoded layouts containing spatial location of pins to be routed are fed into the network, and after 15 fully convolutional layers followed by a comparator, 8 layout layers are produced, which are then decoded to obtain the routed layouts. This work formulates routing as a binary segmentation problem on a per-pixel per-layer basis, where the network is trained to correctly classify pixels in each layout layer to be *on* or *off*. [He and Bao, 2020] models the circuit routing as a sequential decision-making problem, and solve it by Monte Carlo tree search (MCTS) with DNN guided rollout. A recent study [Utyamishv and Partin-Vaisband, 2020] proposes a global router that learns from routed circuits and autonomously routes unseen layouts. Different from traditional routing flow, this approach redefines the global routing as a classical image-to-image processing problem and handles the imaging problem in a unified, single-step non-iterative manner with a deep learning system, comprising a variational autoencoder and a custom loss.

4.3 Reinforcement Learning for Routing

RL is also a promising way to tackle routing, as it can be seen as a process that comprises decision-making phases. A DQN agent [Liao *et al.*, 2020b], as one of the first attempts to combine RL with global routing, learns to decide the routing direction on a 3D grid graph at each step, e.g. traveling north, south, and so on. For detailed routing, [Liao *et al.*, 2020a] presents an attention-based REINFORCE method for obtaining routing orders, followed by a classical pattern router to finish the routing given the order, for small benchmarks with up to thousands nets. [Lin *et al.*, 2021] tackles routing ordering with an asynchronous actor-critic framework for routing millions of nets and improves the solution quality over the state-of-the-art detailed router with policy distillation. [Ren and Fojtik, 2021] employs a genetic algorithm to generate initial routing choices and then uses RL to progressively handle design rule violations in standard cell routing. Regarding rectilinear Steiner minimum tree (RSMT) construction [Hartmanis, 1982], a process that is a fundamental problem in EDA and computer science and typically runs millions of times in traditional global routers, REST [Liu *et al.*, 2021a] is the first attempt to solve RSMT construction using a machine learning-based method. A new concept, rectilinear edge sequence (RES), is proposed to encode an RSMT solution, and an actor-critic model is devised to construct an RSMT.

4.4 Challenges and Limitations for Routing

Despite the existing efforts on learning-based routing, it is difficult for current techniques to systematically outperform classical routing algorithms in both efficiency and solution quality under fair comparison. Most learning-based techniques work well on small circuits with thousands of nets, while a practical routing engine needs to handle millions of nets on an ultra-large 3D grid graph ($> 1000 \times 1000 \times 10$) efficiently and produce high-quality solutions.

5 Open-source and Public Datasets

Open-source (OS). This area is becoming more and more open, and we list a few representative OS examples. Google releases a framework for floorplanning with distributed DRL¹. The end-to-end learning approach DeepPlace for macros and standard cells placement problem is also publicly available². Another project DREAMPlace provides a deep learning toolkit-enabled VLSI placement tool³. The attempt to combine DRL with global routing is now available⁴. We believe that these open-source resources would foster collaborations between academic and industry placement and routing tools.

Datasets. Starting from the pure wirelength-driven formulations in 2005, regular research contests for placement are held every year at ISPD and ICCAD to provide various circuit designs contributed by industry in the common Bookshelf format. From then on, the contests broadened their

specific optimization objectives toward cell density, global routability [Viswanathan *et al.*, 2011], detailed routability [Yutsis *et al.*, 2014], and timing [Kim *et al.*, 2015]. A popular benchmark for both placement and routing is the ISPD 2015 [Bustany *et al.*, 2015]. It consists of five circuits, and each of them has multiple floorplans produced from setting different macro locations with predefined heuristics. ISPD announced two global routing contests in 2007 [Nam *et al.*, 2007] and 2008 [Nam *et al.*, 2008] and two initial detailed routing contests in 2018 [Mantik *et al.*, 2018] and 2019 [Liu *et al.*, 2019], respectively, along with the benchmarks.

6 Conclusion and Outlook

In modern physical design flow, it takes human experts weeks to iterate the placement tools in order to produce solutions with no design rule check violations after routing stage. With rapid development of machine learning, a promising solution for this obstacle is to propose efficient and effective learning framework for solving placement and routing either sequentially or concurrently. [Cheng and Yan, 2021] designs a joint learning approach for either macro placement and routing or placement of macros and standard cells. However, this is not a complete design cycle since millions of or even billions of standard cells are left behind. For future works, learning-based routing solvers merit particular attention as the final building block for the entire end-to-end learning paradigm.

References

- [Agnesina *et al.*, 2020] A. Agnesina, K. Chang, and S. K. Lim. Vlsi placement parameter optimization using deep reinforcement learning. In *ICCAD*, 2020.
- [Agnihotri *et al.*, 2005] A. R. Agnihotri, S. Ono, and P. H. Madden. Recursive bisection placement: Feng shui 5.0 implementation details. In *ISPD*, 2005.
- [Al-Hyari *et al.*, 2021] A. Al-Hyari, H. Szentimrey, A. Shamli, T. Martin, G. Gréwal, and S. Areibi. A deep learning framework to predict routability for fpga circuit placement. *TRETS*, 2021.
- [Alawieh *et al.*, 2020] M. Alawieh, W. Li, Y. Lin, L. Singhal, M. Iyer, and D. Pan. High-definition routing congestion prediction for large-scale fpgas. In *ASP-DAC*, 2020.
- [Alpert *et al.*, 2010] C. Alpert, Z. Li, M. Moffitt, G. Nam, J. Roy, and G. Tellez. What makes a design difficult to route. In *ISPD*, 2010.
- [Batterywala *et al.*, 2002] S. Batterywala, N. Shenoy, W. Nicholls, and H. Zhou. Track assignment: A desirable intermediate step between global routing and detailed routing. In *ICCAD*, 2002.
- [Breuer, 1977] M. A. Breuer. A class of min-cut placement algorithms. In *DAC*, 1977.
- [Bustany *et al.*, 2015] I. Bustany, D. Chinnery, J. Shinnerl, and V. Yutsis. Ispd 2015 benchmarks with fence regions and routing blockages for detailed-routing-driven placement. In *ISPD*, 2015.
- [Caldwell *et al.*, 2000] A. E. Caldwell, A. B. Kahng, and I. L. Markov. Optimal partitioners and end-case placers for standard-cell layout. *TCAD*, 19(11):1304–1313, 2000.

- [Chan *et al.*, 2016] W. Chan, Y. Du, A. Kahng, S. Nath, and K. Samadi. Beol stack-aware routability prediction from placement using data mining techniques. In *ICCD*, 2016.
- [Chan *et al.*, 2017] W. Chan, P. Ho, A. Kahng, and P. Saxena. Routability optimization for industrial designs at sub-14nm process nodes using machine learning. In *ISPD*, 2017.
- [Chang and Lin, 2004] Y. Chang and S. Lin. Mr: A new framework for multilevel full-chip routing. *TCAD*, 2004.
- [Chang *et al.*, 2010] Y.-J. Chang, Y.-T. Lee, J.-R. Gao, P.-C. Wu, and T.-C. Wang. Nthru-route 2.0: a robust global router for modern designs. *TCAD*, 2010.
- [Chen *et al.*, 2008a] H. Chen, Y. Chuang, Y.-W. Chang, and Y. Chang. Constraint graph-based macro placement for modern mixed-size circuit designs. In *ICCAD*, 2008.
- [Chen *et al.*, 2008b] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang. Ntuplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints. *TCAD*, 2008.
- [Chen *et al.*, 2009] H. Chen, C. Hsu, and Y. Chang. High-performance global routing with fast overflow reduction. In *ASP-DAC*, 2009.
- [Chen *et al.*, 2019] G. Chen, C. Pui, H. Li, J. Chen, B. Jiang, and E. Young. Detailed routing by sparse grid graph and minimum-area-captured path search. In *ASP-DAC*, 2019.
- [Chen *et al.*, 2020] J. Chen, J. Kuang, G. Zhao, D. Huang, and E. Young. Pros: A plug-in for routability optimization applied in the state-of-the-art commercial eda tool using deep learning. In *ICCAD*, 2020.
- [Cheng and Yan, 2021] R. Cheng and J. Yan. On joint learning for solving placement and routing in chip design. *NeurIPS*, 2021.
- [Cheng *et al.*, 2018a] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang. Replace: Advancing solution quality and routability validation in global placement. *TCAD*, 2018.
- [Cheng *et al.*, 2018b] W.-K. Cheng, Y.-Y. Guo, and C.-S. Wu. Evaluation of routability-driven macro placement with machine-learning technique. In *ISNE*. IEEE, 2018.
- [Cheon *et al.*, 2005] Y. Cheon, P. Ho, A. Kahng, S. Reda, and Q. Wang. Power-aware placement. In *DAC*, 2005.
- [Cho and Pan, 2007] M. Cho and D. Pan. Boxrouter: A new global router based on box expansion and progressive ilp. *TCAD*, 2007.
- [Chu and Wong, 2007] C. Chu and Y.-C. Wong. Flute: Fast lookup table based rectilinear steiner minimal tree algorithm for vlsi design. *TCAD*, 2007.
- [Cong *et al.*, 2001] J. Cong, J. Fang, and K.-Y. Khoo. Dune-a multilayer gridless routing system. *TCAD*, 2001.
- [Fiduccia and Mattheyses, 1982] C. M. Fiduccia and R. M. Mattheyses. A linear-time heuristic for improving network partitions. In *DAC*. IEEE, 1982.
- [Gao *et al.*, 2008] J.-R. Gao, P.-C. Wu, and T.-C. Wang. A new global router for modern designs. In *ASP-DAC*, 2008.
- [Hamolia and Melnyk, 2021] V. Hamolia and V. Melnyk. A survey of machine learning methods and applications in electronic design automation. In *ACIT*, 2021.
- [Hartmanis, 1982] J. Hartmanis. Computers and intractability: a guide to the theory of np-completeness (michael r. Garey and david s. Johnson). *Siam Review*, 1982.
- [He and Bao, 2020] Y. He and F. S. Bao. Circuit routing using monte carlo tree search and deep neural networks. *arXiv preprint arXiv:2006.13607*, 2020.
- [He *et al.*, 2020] Z. He, Y. Ma, L. Zhang, P. Liao, N. Wong, B. Yu, and M. D. Wong. Learn to floorplan through acquisition of effective local search heuristics. In *ICCD*, 2020.
- [Ho *et al.*, 2004] S.-Y. Ho, S.-J. Ho, Y.-K. Lin, and W.-C. Chu. An orthogonal simulated annealing algorithm for large floorplanning problems. *VLSI Systems*, 2004.
- [Hsu *et al.*, 2011] M.-K. Hsu, S. Chou, T.-H. Lin, and Y.-W. Chang. Routability-driven analytical placement for mixed-size circuit designs. In *ICCAD*, 2011.
- [Huang *et al.*, 2019] Y. Huang, Z. Xie, G. Fang, T. Yu, H. Ren, S. Fang, Y. Chen, and J. Hu. Routability-driven macro placement with embedded cnn-based prediction model. In *DATE*, 2019.
- [Huang *et al.*, 2021] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, et al. Machine learning for electronic design automation: A survey. *TODAES*, 26(5):1–46, 2021.
- [Jain and Okabe, 2017] S. R. Jain and K. Okabe. Training a fully convolutional neural network to route integrated circuits. *arXiv preprint arXiv:1706.08948*, 2017.
- [Kahng and Wang, 2005] A. Kahng and Q. Wang. Implementation and extensibility of an analytic placer. *TCAD*, 2005.
- [Kahng *et al.*, 2018] A. B. Kahng, L. Wang, and B. Xu. Triton-route: An initial detailed router for advanced vlsi technologies. In *ICCAD*, 2018.
- [Kastner *et al.*, 2002] R. Kastner, E. Bozorgzadeh, and M. Sarrafzadeh. Pattern routing: Use and theory for increasing predictability and avoiding coupling. *TCAD*, 2002.
- [Kim *et al.*, 2011] M.-C. Kim, J. Hu, D.-J. Lee, and I. Markov. A simple method for routability-driven placement. In *ICCAD*, 2011.
- [Kim *et al.*, 2015] M. Kim, J. Hu, J. Li, and N. Viswanathan. Iccad-2015 cad contest in incremental timing-driven placement and benchmark suite. In *ICCAD*, 2015.
- [Kirkpatrick *et al.*, 1983] S. Kirkpatrick, D. Gelatt, and M. Vecchi. Optimization by simulated annealing. *Science*, 1983.
- [Lee and Markov, 2012] D. Lee and I. Markov. Obstacle-aware clock-tree shaping during placement. *TCAD*, 2012.
- [Lee, 1961] C. Y. Lee. An algorithm for path connections and its applications. *IRE Transactions on Electronic Computers*, 1961.
- [Li *et al.*, 2018] H. Li, W.-K. Chow, G. Chen, E. F. Young, and B. Yu. Routability-driven and fence-aware legalization for mixed-cell-height circuits. In *DAC*, 2018.
- [Li *et al.*, 2019] H. Li, G. Chen, B. Jiang, J. Chen, and E. F. Young. Dr. cu 2.0: A scalable detailed routing framework with correct-by-construction design rule satisfaction. In *ICCAD*, 2019.
- [Liang *et al.*, 2020] R. Liang, H. Xiang, D. Pandey, L. Reddy, S. Ramji, G.-J. Nam, and J. Hu. Drc hotspot prediction at sub-10nm process nodes using customized convolutional network. In *ISPD*, 2020.
- [Liao *et al.*, 2020a] H. Liao, Q. Dong, X. Dong, W. Zhang, W. Zhang, W. Qi, E. Fallon, and L. B. Kara. Attention routing: track-assignment detailed routing using attention-based reinforcement learning. In *IDETC-CIE*, 2020.
- [Liao *et al.*, 2020b] H. Liao, W. Zhang, X. Dong, B. Poczos, K. Shimada, and L. Burak Kara. A deep reinforcement learning approach for global routing. *Journal of Mechanical Design*, 2020.

- [Lin *et al.*, 2020] Y. Lin, Z. Jiang, J. Gu, W. Li, S. Dhar, H. Ren, B. Khailany, and D. Pan. Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement. *TCAD*, 2020.
- [Lin *et al.*, 2021] Y. Lin, T. Qu, Z. Lu, Y. Su, and Y. Wei. Asynchronous reinforcement learning framework and knowledge transfer for net order exploration in detailed routing. *TCAD*, 2021.
- [Liu *et al.*, 2013] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao. Nctu-gr 2.0: Multithreaded collision-aware global routing with bounded-length maze routing. *TCAD*, 2013.
- [Liu *et al.*, 2019] W. Liu, S. Mantik, W. Chow, Y. Ding, A. Farshidi, and G. Posser. Ispd 2019 initial detailed routing contest and benchmark with advanced routing rules. In *ISPD*, 2019.
- [Liu *et al.*, 2021a] J. Liu, G. Chen, and E. F. Young. Rest: Constructing rectilinear steiner minimum tree via reinforcement learning. In *DAC*. IEEE, 2021.
- [Liu *et al.*, 2021b] S. Liu, Q. Sun, P. Liao, Y. Lin, and B. Yu. Global placement with deep learning-enabled explicit routability optimization. In *DATE*, 2021.
- [Lu *et al.*, 2015] J. Lu, P. Chen, C.-C. Chang, L. Sha, D. J.-H. Huang, C.-C. Teng, and C.-K. Cheng. eplace: Electrostatics-based placement using fast fourier transform and nesterov's method. *TODAES*, 2015.
- [Lu *et al.*, 2021] Y.-C. Lu, S. Pentapati, and S. K. Lim. The law of attraction: Affinity-aware placement optimization using graph neural networks. In *ISPD*, 2021.
- [Luo *et al.*, 2006] T. Luo, D. Newmark, and D. Z. Pan. A new lp based incremental timing driven placement for high performance designs. In *DAC*, 2006.
- [Maarouf *et al.*, 2018] D. Maarouf, A. Alhyari, Z. Abuowaimer, T. Martin, A. Gunter, G. Gréwal, S. Areibi, and A. Vannelli. Machine-learning based congestion estimation for modern fpgas. In *FPL*, 2018.
- [Mantik *et al.*, 2018] S. Mantik, G. Posser, W.-K. Chow, Y. Ding, and W.-H. Liu. Ispd 2018 initial detailed routing contest and benchmarks. In *ISPD*, 2018.
- [Markov *et al.*, 2015] I. Markov, J. Hu, and M. Kim. Progress and challenges in vlsi placement research. *Proc. of IEEE*, 2015.
- [Mirhoseini *et al.*, 2021] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi, et al. A graph placement methodology for fast chip design. *Nature*, 2021.
- [Nam *et al.*, 2002] G.-J. Nam, K. A. Sakallah, and R. A. Rutenbar. A new fpga detailed routing approach via search-based boolean satisfiability. *TCAD*, 2002.
- [Nam *et al.*, 2007] G.-J. Nam, M. Yildiz, D. Pan, and P. Madden. ISPD 2007 Global Routing Contest. 2007.
- [Nam *et al.*, 2008] G.-J. Nam, C. Sze, and M. Yildiz. The ispd global routing benchmark suite. In *ISPD*, 2008.
- [Nieberg, 2011] T. Nieberg. Gridless pin access in detailed routing. In *DAC*. IEEE, 2011.
- [Ozdal and Wong, 2009] M. Ozdal and M. Wong. Archer: A history-based global routing algorithm. *TCAD*, 2009.
- [Ozdal, 2009] M. M. Ozdal. Detailed-routing algorithms for dense pin clusters in integrated circuits. *TCAD*, 2009.
- [Pan *et al.*, 2012] M. Pan, Y. Xu, Y. Zhang, and C. Chu. Fastroute: An efficient and high-quality global router. *VLSI Design*, 2012.
- [Pui *et al.*, 2017] C.-W. Pui, G. Chen, Y. Ma, E. F. Young, and B. Yu. Clock-aware ultrascale fpga placement with machine learning routability prediction. In *ICCAD*, 2017.
- [Qi *et al.*, 2014] Z. Qi, Y. Cai, and Q. Zhou. Accurate prediction of detailed routing congestion using supervised data learning. In *ICCD*, 2014.
- [Rapp *et al.*, 2021] M. Rapp, H. Amrouch, Y. Lin, B. Yu, D. Z. Pan, M. Wolf, and J. Henkel. Mlcad: A survey of research in machine learning for cad keynote paper. *TCAD*, 2021.
- [Ren and Fojtik, 2021] H. Ren and M. Fojtik. Standard cell routing with reinforcement learning and genetic algorithm in advanced technology nodes. In *ASP-DAC*, 2021.
- [Riess and Ettelt, 1995] B. Riess and G. Ettelt. Speed: Fast and efficient timing driven placement. In *ISCAS*, 1995.
- [Shin and Sangiovanni-Vincentelli, 1987] H. Shin and A. Sangiovanni-Vincentelli. A detailed router based on incremental routing modifications: Mighty. *TCAD*, 1987.
- [Shunmugathammal *et al.*, 2020] M. Shunmugathammal, C. Columbus, and S. Anand. A novel b* tree crossover-based simulated annealing algorithm for combinatorial optimization in vlsi fixed-outline floorplans. *Circuits, Systems, and Signal Processing*, 2020.
- [Spindler *et al.*, 2008] P. Spindler, U. Schlichtmann, and F. Johannes. Kraftwerk2-a fast force-directed quadratic placement approach using an accurate net model. *TCAD*, 2008.
- [Sun *et al.*, 2018] F. Sun, H. Chen, C. Chen, C. Hsu, and Y. Chang. A multithreaded initial detailed routing algorithm considering global routing guides. In *ICCAD*, 2018.
- [Tabrizi *et al.*, 2018] A. F. Tabrizi, L. Rakai, N. K. Darav, I. Boustany, L. Behjat, S. Xu, and A. Kennings. A machine learning framework to identify detailed routing short violations from a placed netlist. In *DAC*, 2018.
- [Taghavi *et al.*, 2005] T. Taghavi, X. Yang, and B. Choi. Dragon2005: Large-scale mixed-size placement tool. In *ISPD*, 2005.
- [Utyamishev and Partin-Vaisband, 2020] D. Utyamishev and I. Partin-Vaisband. Late breaking results: A neural network that routes ics. In *DAC*, 2020.
- [Vashisht *et al.*, 2020] D. Vashisht, H. Rampal, H. Liao, Y. Lu, D. Shanbhag, E. Fallon, and L. B. Kara. Placement in integrated circuits using cyclic reinforcement learning and simulated annealing. *arXiv preprint arXiv:2011.07577*, 2020.
- [Viswanathan *et al.*, 2011] N. Viswanathan, C. Alpert, C. Sze, Z. Li, G. Nam, and J. Roy. The ispd-2011 routability-driven placement contest and benchmark suite. In *ISPD*, 2011.
- [Ward *et al.*, 2012] S. Ward, D. Ding, and D. Z. Pan. Pade: A high-performance placer with automatic datapath extraction and evaluation through high-dimensional data learning. In *DAC*, 2012.
- [Wu *et al.*, 2009] T. Wu, A. Davoodi, and J. Linderth. Grip: Scalable 3d global routing using integer programming. In *DAC*, 2009.
- [Wu *et al.*, 2010] T. Wu, A. Davoodi, and J. Linderth. A parallel integer programming approach to global routing. In *DAC*, 2010.
- [Xie *et al.*, 2018] Z. Xie, Y. Huang, G. Fang, H. Ren, S. Fang, Y. Chen, and J. Hu. Routenet: Routability prediction for mixed-size designs using convolutional neural network. In *ICCAD*, 2018.
- [Xu *et al.*, 2016] X. Xu, B. Yu, J.-R. Gao, C.-L. Hsu, and D. Z. Pan. Parr: Pin-access planning and regular routing for self-aligned double patterning. *TODAES*, 2016.

- [Yoshimura and Kuh, 1982] T. Yoshimura and E. S. Kuh. Efficient algorithms for channel routing. *TCAD*, 1982.
- [Yu and Zhang, 2019] C. Yu and Z. Zhang. Painting on placement: Forecasting routing congestion using conditional generative adversarial nets. In *DAC*, 2019.
- [Yutsis *et al.*, 2014] V. Yutsis, I. Bustany, D. Chinnery, J. Shinnerl, and W. Liu. Ispd 2014 benchmarks with sub-45nm technology rules for detailed-routing-driven placement. In *ISPD*, 2014.
- [Zhang and Chu, 2011] Y. Zhang and C. Chu. Regularroute: An efficient detailed router with regular routing patterns. In *ISPD*, 2011.
- [Zhou *et al.*, 2015] Q. Zhou, X. Wang, Z. Qi, Z. Chen, Q. Zhou, and Y. Cai. An accurate detailed routing routability prediction model in placement. In *ASQED*, 2015.
- [Zhou *et al.*, 2019] Z. Zhou, S. Chahal, T.-Y. Ho, and A. Ivanov. Supervised-learning congestion predictor for routability-driven global routing. In *VLSI-DAT*, 2019.